

Soft Hours

Project Proposal

1. Project Overview

Soft Hours is a story-driven visual novel with a repeatable gameplay loop where the player works as a therapist seeing an endless stream of patients. Through dialogue choices and shared interactions, the player manages each patient's emotional stats across a session, working to keep them within a healthy range before the next patient arrives. The game carries a grounded psychological tone, reflecting how everyday mental health struggles look and feel in real life.

Beyond its mechanics, Soft Hours carries a quiet message that showing up for someone, even imperfectly, matters. Each patient reflects a relatable emotional struggle, and the consequences of neglect are drawn from reality, making every session feel meaningful and human.

2. Project Review

Soft Hours takes mechanical inspiration from stat-management visual novels such as *NEEDY STREAMER OVERLOAD* and *Volcano Princess*, borrowing the core idea of managing a character's emotional wellbeing through choices that compound over time. However, unlike its references which focus on a single character, Soft Hours introduces five distinct patients each tied to a different mental health struggle, keeping the experience varied across the full run. The game shifts the tone away from dark or satirical themes toward something grounded and empathetic, reflecting everyday emotional struggles that are quietly relatable. Its key improvement over existing titles is the repeatable loop structure inspired by *That's Not My Neighbor*, where randomized patient information and dialogue keep every session feeling fresh without requiring entirely new content.

3. Programming Development

3.1 Game Concept

Soft Hours is a turn-based visual novel where the player steps into the role of a therapist managing an endless stream of patients, each carrying a different emotional struggle. Overthinking, Anger Issues, Depression, Trauma, and Burnout. The gameplay loop is short and repeatable. A patient arrives with a randomized name, background, and emotional state, the player works through a dialogue session with them, and the outcome depends on how well the player managed their stats.

Each patient session runs through a set of dialogue exchanges where the player picks from 3 to 4 response choices, each carrying hidden stat values. Every patient shares 4 core emotional stats, Hope, Calm, Trust, and Motivation, alongside 1

unique stat tied to their specific illness. Two additional pressure meters, Exhaustion and Loneliness, must be kept in check throughout the session. If stats drift too low the consequences reflect reality. Some patients shut down and walk away, others become volatile. The player carries 3 hearts across the full run. Losing a patient costs a heart, and reaching zero ends the game.

The dialogue system is structured around 5 illness types, each with 5 possible emotional states. Each emotional state has dialogue sets per sprite, with each set containing 3 to 4 player choices, ensuring variety without requiring entirely new content every run. Sprites serve as visual placeholders that express emotions matching the current dialogue state, while the patient's name, age, and background are randomized each visit.

3.2 Object-Oriented Programming Implementation

The following classes will be implemented in Soft Hours:

Class	Role	Key Attributes	Key Methods
Game	Master controller. Manages the main loop, screen, and state transitions.	screen, clock, state, hearts	run(), change_state(), check_gameover()
Patient	Abstract base class for all patients. Holds shared stats and core session logic.	name, age, illness, stats: dict, unique_stat	update_stat(), check_stat_range(), get_expression()
OverthinkingPatient	Inherits Patient. Adds Clarity as unique stat.	clarity	on_stat_fail(), get_dialogue_set()
AngerPatient	Inherits Patient. Adds Control stat. Triggers instant game over on failure.	control	on_stat_fail(), get_dialogue_set()
DepressionPatient	Inherits Patient. Adds Presence stat. Patient goes silent and walks away.	presence	on_stat_fail(), get_dialogue_set()
TraumaPatient	Inherits Patient. Adds Stability stat. Patient shuts down and flees.	stability	on_stat_fail(), get_dialogue_set()
BurnoutPatient	Inherits Patient. Adds Energy stat. Patient collapses and becomes unresponsive.	energy	on_stat_fail(), get_dialogue_set()
DialogueManager	Manages dialogue flow, loads dialogue sets, picks random sentences, tracks choices.	current_set, illness_type, emotion_state	load_dialogue(), pick_random(), apply_choice()

StatSystem	Reads, updates, and monitors all stats. Triggers warnings when values leave safe range.	stats, thresholds, warning_flags	update(), check_range(), trigger_warning()
DataLogger	Records every interaction and stat change per turn to a CSV file for later analysis.	session_id, log_path, turn_count	log_turn(), log_session_end(), export_csv()

3.3 Algorithms Involved

Rule-Based Stat Logic

Each dialogue choice carries a hidden dictionary of stat modifiers. When a choice is made the StatSystem applies these modifiers to the patient's current stats and evaluates whether any value has left the safe threshold range. This is a rule-based evaluation that runs every turn.

Randomization System

At the start of each patient visit the game randomly selects a name, age, background information, illness type, emotional state, and sprite. Within the dialogue system, sentences are randomly pulled from the appropriate set matching the illness and emotional state, ensuring variety across sessions without requiring entirely new content.

Event-Driven Input Handling

The game runs on Pygame's event-driven architecture. Player input triggers choice selection events which feed into the dialogue and stat systems. UI transitions, warning displays, and patient departure animations are all handled through event queues.

Consequence Resolution

At the end of each session the game evaluates the patient's final stat state against their illness type and determines the outcome. Each illness has its own consequence logic, so the same stat failure can produce different results depending on who the patient is.

4. Statistical Data (Prop Stats)

4.1 Data Features

The following features will be tracked per turn throughout each session. With 20 or more turns per patient across multiple sessions, the dataset will comfortably exceed 100 rows per feature. The DataLogger operates in append mode so that data from multiple runs, including short ones, accumulates automatically in the same CSV file.

Feature	Why collect this / what can it be used for	How to obtain 100 values	Which variable / which class collects it	How will it be displayed
Turn Number	Tracks the timeline of each session. Used to align all other features to a point in time and measure session length.	Logged every turn automatically. 20 turns per patient, 5+ patients per run easily exceeds 100.	turn_count in DataLogger	X-axis for all time-series graphs
Patient Illness	Identifies which illness type is being handled. Used to compare difficulty and outcomes across illness categories.	Assigned at session start, repeated each turn row. Accumulates across multiple patient visits.	illness attribute in Patient	Category axis in bar graphs and pie chart
Stat Values	Tracks all 6 emotional stats per turn. Used to visualize how player choices affect patient wellbeing over time.	All 6 stats written every turn. 20 turns x 6 stats = 120 values per session minimum.	stats dict in StatSystem	Line graph (stat trends over turns)
Warning Triggered	Records whether a stat hit the danger boundary. Used to identify which stats are hardest to manage and which illness is most demanding.	Boolean flag written every turn. Accumulates naturally across all sessions.	warning_flags in StatSystem	Bar graph (warnings per stat) and Correlation Heatmap
Time Per Turn	Measures how long the player took to decide each turn. Used to analyze difficulty and hesitation patterns per illness type.	Timestamp difference calculated on player confirm. Every turn produces one value.	Calculated in Game, logged by DataLogger	Boxplot (decision time distribution per illness)
Session Outcome	Records the final result of each patient session. Used to measure overall player performance and success rate per illness.	Written at session end, back-filled to all turn rows for that session. Every session produces one outcome value.	Evaluated in Game, stored by DataLogger	Bar graph (outcome count per illness type)
Session Score	Cumulative score per session (+10 success, -5 heart lost, -2 warning). Used to track player improvement over multiple runs.	Calculated at session end from aggregated turn data. One score per session, accumulates across playthroughs.	Calculated in Game, logged by DataLogger	Line graph (score trend over sessions)

4.2 Data Recording Method

All statistical data will be saved to a CSV file using Python's built-in csv module combined with the Pandas library. The DataLogger class writes one row per turn in append mode, capturing all tracked features at the moment they occur. Session Outcome and Session Score are written as pending placeholder values each turn and updated to their final values when the session closes, ensuring the row structure remains consistent. A new session ID is generated at the start of each run, allowing multiple playthroughs to accumulate in the same file and be compared during analysis.

4.3 Data Analysis Report

The recorded CSV data will be loaded and analyzed using Pandas and visualized in a Tkinter statistics panel that opens after each session ends. The analysis will include the following:

- Stat trend lines per session showing how each emotional stat changed turn by turn, displayed as a line chart.
- Most neglected stat across all sessions, calculated from the frequency of warning triggers per stat, displayed as a bar chart.
- Average decision time per illness type, showing which patients caused the most player hesitation, displayed as a boxplot.
- Session outcome distribution showing the ratio of successful sessions versus losses per illness type, displayed as a bar graph.
- Relationship between warning frequency and session outcome displayed as a correlation heatmap.
- Session score over time showing whether the player is improving across runs, displayed as a line graph.

Statistical measures used will include mean, median, frequency counts, and min/max per feature. All charts will be rendered using Matplotlib embedded within the Tkinter panel.

Graph Planning

Graph	Feature Name	Graph Objective	Graph Type	X-axis	Y-axis
Graph 1	Stat Values	Show how each emotional stat changes turn by turn within a session to visualize	Line Graph (Time-series)	Turn Number	Stat Value (0-100)

		the effect of player choices over time.			
Graph 2	Warning Triggered	Show which stats triggered the most warnings across all sessions to identify the hardest stats to manage.	Bar Graph (Distribution)	Stat Name	Warning Count
Graph 3	Time Per Turn	Show the spread of decision times per illness type to understand which patients cause the most hesitation.	Boxplot (Distribution)	Illness Type	Seconds Per Turn
Graph 4	Session Outcome	Show how many sessions ended in success vs failure per illness type to compare difficulty across conditions.	Grouped Bar Graph (Proportion)	Illness Type	Session Count
Graph 5	Warning Triggered vs Session Outcome	Show correlations between stat neglect and session outcomes to reveal which stats matter most.	Correlation Heatmap (Relation)	Stat Name	Outcome Category

5. Project Planning Timeline

Week	Week (Alt)	Task
1	8	Proposal submission and project initiation
2	9	Core architecture setup, Game, Patient base class, StatSystem, DataLogger
3	10	Patient subclasses, dialogue structure, randomization system
4	11	Dialogue content writing, sprite expressions, UI layout in Pygame
5	12	Full gameplay loop testing, stat balancing, consequence logic
6	13	CSV logging, Pandas integration, Tkinter statistics panel
7	14	Final polish, bug fixing, and draft submission

6. Document Version

Version	Date	Editor
1.2	28 March 2026	Apichai Pattanakamolkul

Date	Name	Feedback
30/03	Peraya	You should clean your proposal. Format, Grammar, etc. to improve readability. However, you have a Graph Plan and a Feature Table, but you are missing the Table of Statistical Values (showing Mean, Median, Max, Min, etc.). The rubric requires at least 1 table specifically for showing statistical values.